

시스템분석 및 설계 보고서

컴퓨터소프트웨어공학과

2026. 6. 17.

프로젝트 명	현대적 저수준 그래픽스 API 기반 경량 플랫폼 네이티브 렌더링 프레임워크
팀명	DyD (Direct, Yield, Dynamic media)

구분	학번	이름
팀장	20242215	한재승
팀원	20240901	윤 훈
	20242207	정현진
	20240846	정준혁
지도교수		정석용

목 차

I. 프로젝트 개요	
1. 개발 배경 및 필요성	3
2. 개발 내용	5
3. 팀 구성 및 역할	6
II. 관련 작품 분석	7
1. 관련 작품 설명	
2. 차별성 및 우수성	
III. 관련 기술 연구(개인별)	
1. 현대 멀티코어 하드웨어 친화적(Hardware-Friendly) 아키텍처에 관한 연구 . 9	
2. DirectX 12 백엔드 및 셰이더 구현	13
3. Metal 백엔드 및 렌더링 구현	15
4. Vulkan 백엔드 및 빛·그림자 구현	17
IV. 시스템 설계	
1. 개발 환경, 도구 및 언어	20
2. 기능 설계	21
2.1 데이터 지향(DOD, Data-Oriented Design) 메모리 레이아웃	21
2.2 RHI (Rendering Hardware Interface)	21
2.3 Multi-Threading 및 Task Scheduling	22
2.4 렌더링 파이프라인 및 Render Graph	23
2.5 GPU 연산 확장 (Compute Shader, Mesh Shader, Ray Tracing)	24
2.6 설계 원칙	25
V. 개발일정 및 역할	
1. 기능별 진척 사항	26
2. 개인별 자기평가(수행내용 및 문제해결 내용)	
2.1 (한재승-20242215)	27
2.2 (윤 훈 -20240901)	28
2.3 (정현진-20242207)	31
2.4 (정준혁-20240846)	34
VI. 참고자료	37

I. 프로젝트 개요

1. 개발 배경 및 필요성

1) 레거시 객체 지향(OOP) 아키텍처의 한계

대다수 상용 게임, 렌더링 엔진은 과거 프로젝트와의 호환을 위해 객체 지향(OOP) 기반 씬 그래프와 두꺼운 추상화 계층을 유지합니다. 다중 상속과 가상 함수 중심의 구조는 객체를 메모리에 흩어 놓아, 런타임에 포인터 추적(pointer chasing)과 가상 호출을 반복하게 만듭니다. 이러한 접근 패턴은 CPU 캐시 미스를 늘려, 코어 수가 늘어도 처리량이 비례해 오르지 못하는 원인이 됩니다. (캐시 미스가 성능에 미치는 원인과 측정은 III.1에서 다룹니다.)

아래의 사진은 OOP 구조가 가지는 가상함수 테이블 구조와 그에 따른 숨겨진 비용들이 어떻게 캐시 미스를 유발하는지에 대한 컨퍼런스의 한 장면입니다.

cppcon | 2018
THE C++ CONFERENCE • BELLEVUE, WASHINGTON

SetNeedsStyleRecalc

Miss!

Miss!

Miss!

Miss!

SetNeedsStyleRecalc

STOYAN NIKOLOV

OOP is dead, long live
Data-oriented design

CppCon.org

CppCon 2018 | @stoyanink 23

2) 현대 그래픽스 API의 하드웨어 제어 철학

DirectX 12와 Vulkan은 기존 드라이버가 상당 부분 추적하던 리소스 상태와 동기화 책임을 애플리케이션으로 옮긴 명시적 그래픽스 API입니다. Metal 역시 command buffer/encoder와 pipeline state 중심의 낮은 오버헤드 제어 모델을 제공하여, 개발자가 GPU 작업의 구성과 제출 흐름을 더 직접적으로 다루게 합니다. 이러한 API의 공통된 방향은 드라이버가 숨겨주던 비용을 줄이는 대신, 엔진이 리소스 수명, 배리어, 파이프라인 상태, 커맨드 기록 방식을 스스로 설계하도록 요구한다는 점입니다.

설계 철학	핵심 메커니즘	기대 효과
상태 객체의 불변화	Pipeline State Object 베이킹(Baking)	런타임 유효성 검사 비용 및 파이프라인 스톱(Stall) 제거
명시적 메모리 배열화	모든 리소스 뷰의 1차원 연속 배열화	포인터 체이싱(Pointer Chasing) 폐기 및 데이터 지역성 확보
Lock-Free 멀티 스레딩	워커 스레드별로 독립적인 커맨드, 메모리 할당	병렬 기록 시 전역 Mutex Lock 차단

그러나 명시적 API를 사용하더라도 두꺼운 범용 래퍼나 구형 상태 기계식 추상화를 없으면, API가 제공하는 직접 제어와 멀티 스레드 기록의 이점이 응용 단계(렌더링 파이프라인의 단계 중 하나)까지 충분히 드러나지 않을 수 있습니다. 따라서 본 프로젝트는 레거시 호환을 우선하는 범용 래퍼보다, 하드웨어 캐시 구조와 명시적 자원 모델에 맞춘 얇은 데이터 지향 프레임워크(DOD, Data-Oriented Design)를 목표로 합니다.

이 목표를 구체화하기 위해 다음 절에서는 프레임워크의 계층 구조, 사용자 시나리오, RHI와 렌더링 파이프라인의 개발 내용을 설명합니다.

2. 개발 내용

1) 시스템 아키텍처 구성도

앞 절에서 제시한 목표를 구현하기 위해 본 프레임워크는 편의 기능을 하나의 두꺼운 객체 계층에 모으지 않고, 데이터 흐름과 제어 책임에 따라 네 계층으로 분리합니다. 하위 Foundation은 캐시 친화적 데이터 처리와 멀티스레딩 기반을 제공하고, RHI는 Vulkan·D3D12·Metal의 명시적 자원 모델을 얇게 노출합니다. 그 위의 Render Graph는 패스 간 리소스 수명과 배리어를 관리하며, Render Front-End는 사용자가 장면과 재질을 다루는 상위 진입점 역할을 합니다.

Application / Examples 사용자 코드와 샘플 진입점		계층 공통 설계 요소 Multi-Threading Job → 스레드별 기록 Frame Indexing 더블/트리플 버퍼링
Layer 4 Render Front-End	Renderer · Scene/Material · Flattened Render Queue Bindless 바인딩과 상위 렌더링 진입점	
Layer 3 Render Graph	Pass 선언 · 리소스 수명/배리어 자동화 Transient 풀과 멀티스레드 기록 연계	
Layer 2 RHI	IDevice · ICommandList(Thread-Local) · Resources Backends: Vulkan · D3D12 · Metal · Null	
Layer 1 Foundation	Math(SIMD) · Platform · Core · Job/Task System 캐시 친화적 데이터 처리와 실행 기반	

dy_engine 계층형 아키텍처와 계층 공통 설계 요소

2) 시스템 서비스 시나리오

본 프레임워크의 사용자는 상위 렌더러를 이용해 기본 렌더링 경로를 구성할 수 있으며, 필요한 경우 RHI까지 내려가 API 별 세부 동작을 직접 제어할 수 있습니다. 일반적인 사용 흐름은 플랫폼 선택, 리소스 생성, 렌더 패킷 구성, 커맨드 기록 및 제출 순서로 이루어집니다.

멀티 플랫폼 실행의 경우, 사용자는 'IDevice::Create'를 통해 실행 환경에 맞는 백엔드(Vulkan·D3D12·Metal)를 선택합니다. 이후 버퍼·텍스처 생성, 파이프라인 설정, 드로우 호출은 공통 인터페이스를 통해 수행되므로, 상위 렌더링 코드는 API 호출에 직접 의존하지 않습니다.

성능 조정이 필요한 경우에는 제어 깊이를 단계적으로 선택합니다. 기본 경로는 'RendererDesc'로 바인딩 방식, 포맷, 조명, 그림자 설정을 조정하고, 더 낮은 수준의 제어가 필요할 때는 상위 렌더러의 저수준 API나 'IDevice'·'ICommandList'를 직접 사용합니다. 이 구조는 추상화를 고정된 장벽으로 두지 않고, 기능 구현 단계와 성능 분석 단계에서 필요한 만큼 낮은 계층으로 접근할 수 있게 합니다.

대규모 오브젝트 렌더링에서는 렌더 패킷을 평탄화하고 정렬하여 상태 변경과 드로우 제출 비용을 줄입니다. 리소스 바인딩은 백엔드별 디스크립터/argument 리소스 테이블과 소량 상수 전달 경로로 추상화하여, 다수의 텍스처와 버퍼를 일관된 방식으로 참조하도록 설계합니다.

멀티코어 활용은 현재 설계·진행 중인 영역입니다. 워커 스레드별 독립 커맨드 리스트와 할당자를 두어 커맨드 기록 단계의 전역 경합을 줄이고, 최종 제출 시에는 정렬된 렌더 패킷과 의존성 정보를 기준으로 실행 순서를 확정하는 구조를 목표로 합니다.

따라서 본 프레임워크는 편의 기능을 제공하면서도, 필요한 경우 사용자가 구성 설정, 상위 렌더러의 저수준 API, RHI 직접 사용 순으로 제어 범위를 넓힐 수 있도록 설계합니다. 이러한 점진적 제어 방식이 기존 렌더링 라이브러리와 어떤 차이를 갖는지는 다음 장에서 비교합니다.

3. 팀 구성 및 역할

구분	학번	이름	담당 역할 및 개발 분야
팀장	20242215	한재승	· 엔진 코어 설계, RHI 인터페이스 조합 담당 · 다방면의 최적화 연구와 구현
팀원	20240901	윤 훈	· Vulkan과 Linux 계열 담당 · Post-Processing 연구와 구현
팀원	20242207	정현진	· DirectX 12와 MSVC 계열 담당 · Mesh 관련 연구와 구현
팀원	20240846	정준혁	· Metal과 MacOS 계열 담당 · CPU/GPU Profiling 연구와 구현

II. 관련 작품 분석

1. 관련 작품 설명

1) bgfx

bgfx는 광범위한 플랫폼과 그래픽스 API를 단일 인터페이스로 묶어 제공하는, 사실상 업계 표준급으로 널리 쓰이는 렌더링 라이브러리입니다. 공식 지원 API는 Direct3D 11·12, Metal, OpenGL, OpenGL ES, Vulkan, WebGL, WebGPU 등을 포함하며, 서로 다른 API 차이를 공통 인터페이스로 흡수하는 데 강점이 있습니다. 다만 이러한 범용성은 응용 코드가 네이티브 D3D12/Vulkan 수준의 리소스 상태, 배리어, 디스크립터 바인딩 정책을 직접 다루기보다 bgfx가 제공하는 추상화와 내부 변환에 의존한다는 뜻이기도 합니다. 따라서 본 프로젝트가 목표로 하는 얇은 RHI와 세부 제어 경로를 기준으로 보면, bgfx는 이식성과 안정성을 우선한 범용 렌더링 계층에 가깝습니다.

2) Methane Kit

Methane Kit은 DirectX 12·Vulkan·Metal 등 최신 네이티브 그래픽스 API 위에서 동작하는 C++20 기반 크로스플랫폼 프레임워크입니다. 공식 설명에서도 객체 지향 중간 수준 그래픽스 API를 제공한다고 밝히며, 사용 편의성과 플랫폼 독립성을 중요한 목표로 둡니다. 이 방향은 최신 API를 대상으로 한다는 점에서는 본 프로젝트와 유사하지만, 추상화의 중심이 객체 지향 RHI 인터페이스에 있다는 점에서 차이가 있습니다. 본 프로젝트는 기능 편의보다 얇은 데이터 컨테이너, 명시적 자원 모델, 필요 시 RHI 직접 접근이 가능한 점진적 제어를 우선합니다.

2. 차별성 및 우수성

앞 절에서 제시한 점진적 제어(디폴트→config→상위 우회→RHI 직접 제어), 단일 코드베이스 이식성, 명시적 자원 모델의 직접 노출은 위 두 라이브러리가 공통으로 갖는 한계, 곧 고정된 추상화 깊이에 대응합니다. 주요 차이를 요약하면 다음과 같습니다.

항목	bgfx	Methane Kit	본 프레임워크
대상 API	레거시~최신 광범위	최신 명시적 (DX12, Vulkan, Metal)	최신 명시적 (DX12, Vulkan, Metal)
추상화 방식	범용 렌더링 API	객체 지향 중간 수준 RHI	얇은 데이터 컨테이너
리소스 바인딩 제어	공통 API 내부에서 관리	객체 인터페이스로 추상화	백엔드별 자원 모델을 얇게 노출
바인딩 전략	라이브러리 추상화 중심	RHI 객체 모델 중심	PerDraw, Batched, Bindless 교체
제어 깊이(우회 경로)	네이티브 API 세부 제어 노출 제한	객체 지향 RHI 계층 중심	디폴트→config→상위 우회→RHI 직접
설계 우선순위	호환성 · 이식성	현대성 + 사용 편의	처리량, 하드웨어 제어

표에서 보듯 bgfx는 다양한 플랫폼과 API를 안정적으로 포괄하는 범용 렌더링 계층에 강점이 있고, Methane Kit은 최신 API 위에서 객체 지향 RHI를 제공하여 사용 편의성을 높이는 방향에 가깝습니다. 반면 본 프레임워크는 모든 API를 포괄하는 범용성보다, 최신 명시적 API의 자원 모델과 커맨드 기록 방식을 상위 렌더링 파이프라인까지 비교적 얇게 전달하는 데 초점을 둡니다. 이를 위해 디폴트 설정, config 조정, 상위 렌더러의 저수준 API, RHI 직접 사용의 여러 접근 깊이를 제공하는 점진적 제어 구조를 둡니다.

요약하면 본 프레임워크의 차별성은 '모든 API를 포괄하는 범용성'이나 '객체 지향 중간 수준 API의 편의성'이 아니라, 최신 명시적 API의 이점을 가리지 않는 얇고 데이터 지향적인 설계에 있습니다. 이는 호환성과 이식성을 우선하는 bgfx, 사용 편의성을 강조하는 Methane Kit과 구분되는 지점입니다.

Ⅲ. 관련 기술 연구

1. 현대 멀티코어 하드웨어 친화적(Hardware-Friendly) 아키텍처에 관한 연구

학번	작성자	관련 기술
20242215	한재승	CPU 캐시 아키텍처(L1/L2/L3) 분석 및 물리적 메모리 레이아웃에 관한 연구
		데이터 지향 설계에 입각한 SoA(Structures of Arrays) 구조에 관한 연구
		SIMD 인트린직(Intrinsics)을 활용한 데이터 병렬 처리 성능 측정

대규모 객체 순회나 썬 업데이트처럼 많은 데이터를 반복 접근하는 CPU 작업에서는 연산량보다 메모리 접근 지연이 병목이 되기 쉽습니다. 먼저 CPU 캐시 계층과 캐시 라인 단위의 데이터 이동을 설명하고, 그 위에서 데이터 지향 설계와 SoA 구조가 왜 효과적인지 다룹니다. 이어서 SIMD 인트린직을 이용한 데이터 병렬 처리 방식을 정리하고, 본 프로젝트 코드에서 AoS/SoA 및 SSE 적용 효과를 측정합니다.

본 절은 데이터 배치와 CPU 실행 특성을 설명하기 위해 다음 자료들을 근거로 삼습니다.

선행 연구	관점	기여
Drepper (2007)	하드웨어 메커니즘	캐시 계층과 미스 비용의 원리 — '왜 메모리가 병목인가'
Acton (2014)	엔진 설계 관점	데이터 변환 중심 사고와 캐시 친화적 배치의 필요성
Albrecht (2009)	게임 도메인 실측	OOP 썬 그래프 순회의 L2 미스와 프레임 시간 (19.2→3.3ms) 측정
Nikolov (2018)	현대 패러다임	OOP의 숨은 비용과 데이터 지향 설계를 일반화한 업계 관점

Drepper(2007)는 캐시 계층과 주 메모리 접근 비용을 하드웨어 관점에서 설명하고, Acton(2014)과 Nikolov(2018)의 CppCon 발표는 게임 엔진 설계에서 데이터 배치와 캐시 지역성이 성능에 미치는 영향을 강조합니다. 본 절은 이러한 관점을 바탕으로 데이터를 캐시 친화적으로 배치하는 설계 원리를 정리하고, 본 프로젝트 코드에서 그 효과를 측정합니다.

1) CPU 캐시 아키텍처 및 물리적 메모리 레이아웃

메모리는 단일 속도가 아니라 계층(L1-L2-L3-주 메모리)으로 구성되며, 하위로 갈수록 용량은 크지만 지연이 급격히 늘어납니다. 한 번의 미스로 주 메모리까지 내려가면 수백 사이클, 곧 수백 개 명령을 실행할 시간을 대기로 소모할 수 있습니다 (Drepper, 2007). 데이터는 바이트 단위가 아니라 CPU 캐시 라인 단위로 캐시에 적재됩니다.

계층	대략적 접근 지연	비고
L1	약 4 사이클	
L2	약 12 사이클	
L3	약 40 사이클	
주 메모리	약 100-300 사이클	1회 미스 = 수백 명령 분량 대기

(대략치이며 CPU에 따라 다릅니다. 출처: Drepper, 2007.)

객체 지향 썬 그래프처럼 포인터로 흩어진 객체를 따라가는 접근(pointer chasing)과 가상 함수 테이블 간접 호출은 때 접근이 미스가 되기 쉽습니다. Albrecht(2009)는 OOP 썬 그래프 순회에서 객체당 다수의 L2 미스를 측정하고, 데이터 지향 재배포치로 프레임 시간을 19.2ms에서 3.3ms로 단축한 사례를 보고합니다.

2) 데이터 지향 설계에 입각한 SoA(Structures of Arrays) 구조에 관한 연구

캐시 계층의 관점에서 보면 중요한 것은 “필요한 데이터가 같은 캐시 라인 안에 얼마나 조밀하게 모여 있는지” 입니다. 객체 지향 엔진에서 흔히 나타나는 객체 단위 AoS 배치는 한 객체의 여러 필드를 함께 저장합니다. 그러나 특정 필드만 순회하는 작업(예: 위치 갱신)에서는 캐시 라인에 함께 들어온 나머지 필드가 사용되지 않아 라인의 유효 바이트율이 떨어집니다. 반대로 SoA는 같은 필드를 연속 배열로 분리하므로, 해당 작업이 읽는 데이터가 캐시 라인을 더 효율적으로 채웁니다. 이러한 포인터 추적과 연속 배열의 대비는 데이터 지향 설계의 출발점입니다 (Acton, 2014).

```
// AoS: pos만 갱신해도 vel·mass가 같은 캐시 라인을 차지(낭비)
struct Entity { float3 pos; float3 vel; float mass; /* ... */ };
std::vector<Entity> e;
for (auto& x : e) x.pos += x.vel * dt;

// SoA: pos·vel 배열만 선형 접근 → 라인 유효 바이트율↓, SIMD 적재 용이
std::vector<float3> pos, vel;
for (size_t i = 0; i < n; ++i) pos[i] += vel[i] * dt;
```

3) SIMD 인트린직(Intrinsics)을 활용한 데이터 병렬 처리 성능 측정

SoA는 캐시 지역성뿐 아니라 SIMD 활용에도 유리합니다. 같은 타입의 데이터가 연속으로 놓이면 SIMD 레지스터에 그대로 적재(load)하여 한 명령으로 여러 요소를 처리할 수 있는 반면, AoS는 흩어진 필드를 모으는 gather-shuffle 비용이 들 수 있습니다. 본 측정에서는 SSE의 128비트 XMM 레지스터(`\_\_m128`)를 사용하는 인트린직 구현과 스칼라 구현을 비교했습니다. SSE는 32비트 float 4개를 한 번에 처리할 수 있으므로, 행렬곱처럼 같은 연산이 반복되는 커널에서 효과를 확인하기에 적합합니다.

위 원리의 효과를 본 프로젝트 개발 환경에서 직접 측정했습니다. 표 1은 메모리 레이아웃(AoS/SoA)의 효과를, 표 2는 SIMD의 효과를 봅니다. (표 1은 g++ ‘-O3 -march=native’로 배열 크기를 캐시 용량 안팎으로 키우며 순회 시간을 비교했습니다 — 하드웨어 카운터 대신 이 방식으로 지역성을 관찰. 표 2는 엔진 실제 빌드와 같은 MSVC ‘/O2’로 07_RenderPath에 내장된 행렬 곱 벤치를 측정했습니다.)

표 1. 메모리 레이아웃 - “pos += vel * dt” (float3, scalar, 요소당 ns)

배열 크기		AoS	SoA	SoA 우위
0.5 MB	캐시 내부	1.04 ns	1.07 ns	0.97x
4 MB	L2~L3	1.09 ns	1.09 ns	1.00x
32 MB	L3 초과	5.44 ns	1.20 ns	4.52x
256 MB	주 메모리	5.47 ns	1.39 ns	3.95x

데이터가 캐시에 들어가면 레이아웃 차이가 사실상 없으나($\approx 1\times$), L3를 초과하는 순간 AoS가 4~5배 느려집니다. AoS가 갱신에 쓰지 않는 필드(mass·패딩)까지 캐시 라인째로 끌어와 메모리 대역폭을 낭비하기 때문이며, 따라서 대규모 썬일수록 SoA의 이점이 커집니다.

표 2. SIMD - 'float4x4' 행렬 곱 200만 회

구성	시간	상대
SIMD OFF (scalar)	31.87	1.00x
SIMD ON (SSE)	14.68	2.17x

측정 결과 스칼라 구현은 31.87ms, SSE 구현은 14.68ms로 나타났으며, 같은 행렬 곱 작업에서 약 2.17배의 실행 시간 개선이 확인되었습니다.

다만 이 결과는 행렬 곱처럼 연산 밀도가 높고 데이터 접근 패턴이 규칙적인 커널에서의 측정입니다. 대규모 배열 순회처럼 메모리 대역폭이 먼저 포화되는 작업에서는 SIMD 폭을 넓혀도 같은 비율의 성능 향상을 기대하기 어렵습니다. 따라서 SIMD는 SoA와 같은 연속 메모리 배치와 함께 적용될 때 효과가 가장 분명합니다.

종합하면, 큰 성능 이득은 두 축에서 나옵니다. 대규모 데이터에서는 레이아웃(SoA)이, 연산 집약 커널에서는 SIMD가 결정적입니다. 두 결과 모두 “병목은 연산보다 메모리 접근”이라는 본 절의 논지와 일관됩니다.

2. DirectX 12 백엔드 및 셰이더 구현

학번	작성자	관련 기술
20242207	정현진	DirectX12 백엔드 및 셰이더 구현
DXGI 기반 인프라 및 스왑체인 생성 (DXGI Factory)	• 시스템에 장착된 GPU(Adapter)를 열거 • 화면 출력을 위한 스왑체인 생성	
GPU 하드웨어 및 논리적 디바이스 (Adapter, Device)	• Adapter는 시스템에 장착된 실제 물리적 GPU를 의미 • Device는 GPU와 통신하며 메모리 할당, 리소스 및 뷰 생성 등을 제어하기 위해 생성하는 논리적 객체	
렌더링 명령 기록 및 비동기 실행 큐 (Command List, Command Queue)	• Command List는 CPU가 GPU에서 실행할 렌더링 명령(Draw Call, 상태 변경 등)을 기록하는 객체 • Command Queue는 기록이 완료된 리스트를 제출 받아, GPU가 비동기적 작업을 처리하도록 관리하는 대기열	
스왑체인 (Swapchain)	• 렌더링 될 대상인 Back Buffer 이미지들을 관리 • 프레임 렌더링이 진행되는 동안 화면 깜빡임 없이 부드럽게 다음 화면으로 전환해주는 시스템	
그래픽 리소스 서술자 및 메모리 힙 (Descriptor Heap, View)	• D3D12에서는 텍스처나 버퍼 같은 리소스를 파이프라인에 직접 연결하지 않고 View를 통해 접근 • Descriptor Heap은 View들을 저장하고 관리하는 연속된 메모리 공간	
파이프라인 상태 및 리소스 바인딩 규칙 (PSO, Root Signature)	• 파이프라인 상태 객체(PSO)는 Shader, Blending, Rasterizer 등 그래픽스 파이프라인의 전체 상태를 미리 컴파일하여 묶어둔 객체 • Root Signature는 파이프라인이 텍스처나 상수 버퍼 등 어떤 리소스를 사용할 지 메모리 레이아웃 규칙을 정의	
CPU-GPU 비동기 작업 동기화 (Fence, Event)	• Fence는 독립적으로 동작하는 CPU와 GPU 간의 작업 완료 시점을 맞추기 위한 동기화 객체 • Fence 값을 통해 GPU가 명령 큐에 작업을 어디까지 완료했는지 추적하고, Event 핸들을 통해 CPU가 특정 작업이 끝날 때까지 대기하도록 제어	

1) 사용 기술 개요

DirectX 12(D3D12)는 하드웨어에 대한 명시적 제어와 멀티스레딩 최적화에 강점을 가지는 저수준 그래픽스 API입니다. 본 구현의 목표는 dy_engine의 RHI(Render Hardware Interface) 시스템에 D3D12 백엔드를 연동하고, 화면 출력까지의 렌더링 파이프라인을 구축하는 것입니다.

2) 기술 적용 방식

- 초기화 및 장치 구성: D3D12를 통해 DXGI 팩토리를 생성하고, 최적의 GPU를 탐색하여 논리적 디바이스를 초기화했습니다. 이와 함께 부드러운 화면 출력을 담당하는 스왑체인과 렌더 타겟 뷰(RTV)용 서술자 힙을 구성했습니다.
- 파이프라인 및 리소스 관리: D3D12PipelineState 클래스를 활용해 렌더링 상태와 셰이더 리소스 바인딩 규칙(Root Signature)을 정의했습니다. D3D12Buffer 및 D3D12Texture를 통해 버퍼 생성과 업로드 힙(Upload Heap) 기반의 데이터 전송을 처리했습니다.
- 최신 렌더링 아키텍처 적용: 엔진의 최신 렌더링 구조 개편에 맞춰 D3D12Device 내부에 글로벌 디스크립터 힙(Global Descriptor Heap)을 설계했습니다. 이를 통해 셰이더 내에서 배열 형태로 다수의 텍스처에 한 번에 접근할 수 있는 Bindless Texture 기법을 구현했습니다.
- 명령 처리 흐름: D3D12CommandList 객체를 구현하여 RHI 인터페이스 기반의 드로우 콜, 뷰포트 설정, 리소스 배리어 등의 명령을 할당자에 기록하고, 이를 명령 큐에 제출(ExecuteCommandLists)하여 프레임을 렌더링했습니다.

3. Metal 백엔드 및 렌더링 구현

학번	작성자	관련 기술
20240846	정준혁	Metal 백엔드 및 렌더링 구현

Metal 백엔드 초기화 및 장치 구성 기술

- MTLDevice를 생성하고 GLFW Cocoa 윈도우와 연결되는 CAMetalLayer를 구성하여 Metal 실행 환경을 구성
- MTLCommandQueue를 생성하여 GPU 명령 제출 경로를 구성

렌더링 루프 및 화면 출력 처리 기술

- CAMetalLayer의 nextDrawable을 획득하여 렌더 타겟을 구성하고, CommandBuffer 기록 후 Present하는 기본 렌더링 루프를 구현
- BeginFrame, Submit, Present 흐름을 구성하여 매 프레임 렌더링 경로를 연결

그래픽스 파이프라인 및 셰이더 처리 기술

- MTLRenderPipelineDescriptor를 구성하여 Metal 기반 렌더링 파이프라인을 생성
- MSL(Metal Shading Language) 소스 텍스트를 런타임에 로드하여 newLibraryWithSource로 컴파일하고 파이프라인에 연결

헤더 오염 방지 및 플랫폼 격리 기술

- Pimpl 패턴을 적용하여 Metal Objective-C 타입(id<MTLDevice> 등)을 .mm 구현 파일 내부에 은닉하고 헤더에서 완전히 분리
- '#ifdef ENABLE_METAL' 분기를 통해 Metal 백엔드가 macOS 빌드에서만 컴파일되도록 구성

기본 리소스 생성 및 렌더링 기술

- Vertex Buffer, Index Buffer를 MTLResourceStorageModeShared 방식으로 생성하여 CPU에서 데이터를 업로드하고 GPU가 참조하도록 구성

- MTKTextureLoader를 활용하여 이미지 파일을 GPU 텍스처로 변환하고 Fragment Shader에 바인딩하여 텍스처 렌더링을 구현

1) 사용 기술 개요

Metal은 Apple이 제공하는 저수준 그래픽스 API로, GPU 자원 관리와 렌더링 절차를 개발자가 직접 제어할 수 있도록 설계되었다. 본 작업에서는 dy_engine의 Metal 백엔드를 대상으로, Metal 초기화 과정과 화면 출력 경로를 직접 구성하는 데 초점을 맞추었다. 구체적으로는 MTLDevice 및 CAMetalLayer 초기화, MTLCommandQueue 구성, 렌더 파이프라인 생성, 셰이더 로드, 버퍼 및 텍스처 업로드, 프레임 렌더링 흐름 연결까지의 과정을 하나의 렌더링 경로로 구성하였다.

2) 기술 적용 방식

본 구현에서는 MetalDevice 클래스를 중심으로 Metal 초기화와 프레임 처리를 구성하였다. MetalDevice에서는 MTLDevice 생성, CAMetalLayer 연결, MTLCommandQueue 생성을 수행하여 렌더링에 필요한 기본 장치 구성을 완료하였다. 렌더 파이프라인 구성은 MetalPipeline 클래스에서 수행하였다. MSL 셰이더 소스를 런타임에 텍스트로 읽어 newLibraryWithSource로 컴파일하고, MTLRenderPipelineDescriptor를 구성하여 파이프라인을 생성하였다. 리소스 생성은 MetalBuffer, MetalTexture 클래스를 통해 처리하였다. Shared 모드 버퍼로 Vertex/Index 데이터를 GPU에 업로드하고, MTLTextureDescriptor를 통해 텍스처를 생성하였다.

실제 렌더링 흐름은 BeginFrame → CommandList 기록 → Submit → Present 순서로 구성하였다. MetalCommandList는 렌더패스 설정, 파이프라인 바인딩, PushConstants 전달, DrawInstanced 호출 등의 명령을 담당한다.

헤더 오염 방지를 위해 모든 Metal Objective-C 타입은 Pimpl 패턴으로 .mm 파일 내부에만 존재하도록 격리하였으며, 헤더에는 struct Impl; 전방선언만 노출하였다.

4. Vulkan 백엔드 및 빛·그림자 구현

학번	작성자	관련 기술
20240901	윤 훈	Vulkan 백엔드 및 phong shading을 기반으로 한 빛·그림자 구현
Vulkan 장치 · 스왑체인 · 프레임 동기화	Vulkan 실행 인스턴스, 화면 연결, 물리/논리 장치, 큐 계열, 스왑체인, 화면 출력 이미지를 구성하였다. 프레임 동기화 객체와 프레임별 이미지 사용 상태를 관리하여 이미지 획득, 명령 제출, 화면 출력 순서를 안정화하고, 조명 값과 그림자 행렬이 프레임 사이에서 어긋나며 발생하던 깜빡임 문제를 해결하였다.	
셰이더 빌드 및 데이터 규칙 공통화	CMake와 glslangValidator로 GLSL 셰이더를 SPIR-V로 자동 변환하였다. 공통 셰이더 레이아웃 정의를 통해 C++ 상수, GLSL 바인딩 번호, 텍스처 사용 플래그, 소량 상수 데이터 위치를 공통화하여 셰이더 파일별 하드코딩 의존성을 줄이고 데이터 규칙 불일치 문제를 해결하였다.	
Shadow Pass와 Main Forward Pass 연계	그림자 전용 패스에서 그림자 맵을 생성하고, 메인 렌더 패스에서 해당 그림자 맵과 그림자 행렬 데이터를 GPU 리소스 바인딩 묶음으로 연결하였다. 그림자 생성 여부와 그림자 수신 여부를 분리하여, 그림자를 만드는 대상과 그림자를 받는 대상을 독립적으로 제어하였다.	
빛 · 그림자 셰이더 및 BRDF 계산	장면의 대표 방향광 또는 점광 정보를 조명 상수 데이터로 전달하고, GGX, Smith 기하 감쇠, Schlick Fresnel 기반의 간이 PBR 직접광 계산을 적용하였다. 그림자 가시성을 직접광 항에 반영하여 빛이 닿지 않는 영역이 자연스럽게 어두워지도록 처리하였다.	
RHI 리소스 바인딩과 geometry fetch	OBJ 메시와 채질 정보를 상위 렌더러 경로로 전달하고 GPU 버퍼, 텍스처, 리소스 바인딩 묶음에 연결하였다. 정점/인덱스 저장 버퍼 기반의 셰이더 지오메트리 읽기 구조를 사용하여 draw 정보, 채질, 변환 행렬, 조명/그림자 상수 데이터가 하나의 RHI 렌더링 흐름으로 연결되도록 구성하였다.	

1) 사용 기술 개요

본 파트에서는 Vulkan 백엔드를 단순히 화면 출력용 API 구현체로 다루기보다, 상위 렌더러에서 전달한 조명과 그림자 데이터를 실제 GPU 렌더링 결과로 이어지게 만드는 과정을 중심으로 다루었다. Vulkan은 렌더 패스, 파이프라인 상태, 리소스 바인딩, 동기화 객체를 개발자가 명시적으로 구성해야 하므로, 빛과 그림자 기능을 넣기 위해서는 셰이더 코드뿐 아니라 RHI 리소스 연결, 프레임 동기화, 그림자 맵 생성 경로까지 함께 맞아야 했다.

따라서 구현 범위는 C++17 기반 RHI 인터페이스 구현, Vulkan SDK를 이용한 장치 및 스왑체인 구성, CMake와 glslangValidator를 통한 셰이더 빌드 자동화, GLFW 기반 화면 연결을 포함하되, 최종적으로는 그림자 전용 패스와 메인 렌더 패스를 하나의 렌더링 흐름으로 연결하는 데 초점을 두었다. 특히 공통 셰이더 레이아웃 정의를 분리하여 C++ 코드와 GLSL 셰이더가 같은 바인딩 번호, 텍스처 사용 플래그, 소량 상수 데이터 위치를 공유하도록 만들었고, 이 구조가 조명/그림자 셰이더의 기준점이 되도록 정리하였다.

2) 기술 적용 방식

초기 Vulkan 백엔드에서는 실행 인스턴스, 화면 연결, 물리/논리 장치, 큐 계열, 스왑체인, 화면 출력 이미지를 구성하여 기본 렌더링 기반을 마련하였다. 이후 프레임 동기화 객체와 프레임별 이미지 사용 상태를 관리하여 이미지 획득, 명령 기록, 큐 제출, 화면 출력 순서를 안정화하였다. 이 부분은 단순 프레임 출력뿐 아니라 빛과 그림자에서도 중요했는데, 조명 상수 데이터와 그림자 행렬이 프레임 사이에서 어긋나면 빛이 깜빡이거나 그림자 위치가 순간적으로 흔들려 보였기 때문이다. 프레임 진행 순서와 리소스 바인딩 갱신 흐름을 정리하면서 이 문제가 사라졌고, 조명 방향과 그림자 결과가 프레임마다 일관되게 유지되었다.

렌더링 파이프라인은 메인 렌더 패스와 그림자 패스를 분리하여 구성하였다. 메인 렌더 패스에서는 텍스처, 조명 상수 데이터, 그림자 맵, 그림자 행렬 데이터, 정점/인덱스 저장 버퍼를 GPU 리소스 바인딩 묶음에 연결하고, draw별 변환 행렬과 재질 값은 소량 상수 데이터로 전달하였다. 일반적인 정점 입력 방식에만 의존하지 않고 정점/인덱스 데이터를 저장 버퍼로 넘긴 뒤 셰이더에서 인덱스를 기준으로 읽는 셰이더 기반 지오메트리 읽기 구조를 사용하여, RHI 레벨의 지오메트리 연결을 단순화하고 셰이더 데이터 규칙을 명확히 통제할 수 있게 하였다.

빛과 그림자 처리에서는 장면에 등록된 대표 방향광 또는 점광 정보를 조명 상수 데이터로 변환하고, 그림자 맵 계산 단계에서 얻은 광원 시점 행렬을 그림자 행렬

데이터로 전달하였다. 그림자 생성 여부는 해당 draw가 그림자 패스에 참여할지 결정하고, 그림자 수신 여부는 픽셀 셰이더에서 그림자 맵 샘플링을 적용할지 결정하도록 분리하였다. 조명 계산은 Phong 모델의 확산광/반사광 개념을 참고하여 시작했지만, 상위 Vulkan 셰이더에서는 GGX, Smith 기하 감쇠, Schlick Fresnel을 사용하는 마이크로패킷 기반의 간이 PBR 직접광 모델로 확장하였다. 그림자는 shadow mapping을 기반으로 깊이값 비교, 광원 시점 좌표 변환, 깊이 보정값, PCF 샘플링을 적용하여 self-shadowing, shadow acne, 경계 aliasing을 완화하였다.

IV. 시스템 설계

1. 개발 환경, 도구 및 언어

구분	내역	상세내역
환경	플랫폼	Windows, Linux, MacOS, Android
	컴파일 도구	MSVC, GCC, Clang
	언어	C++17, Object-C++, HLSL, GLSL
	빌드 시스템	CMake
Graphics API	DirectX12	Windows SDK 10
	Vulkan	Vulkan SDK 1.4
	Metal	Metal
도구 라이브러리	윈도우	GLFW
	텍스처	stb_image
	FBX 파싱	ufbx
	glTF 파싱	Fastgltf
	디버그 GUI	IMGUI
서버	플랫폼	Debian Linux
Android	디바이스	

2. 기능 설계

본 절은 앞의 관련 기술 연구와 백엔드 구현 경험을 바탕으로, 엔진 구조를 어떤 기능 단위로 나누어 설계했는지 정리합니다. 설계 방향은 데이터 지향 배치, 저수준 그래픽스 API를 얹게 감싸는 RHI, 멀티코어 환경을 고려한 실행 모델, 향후 Compute Shader, Mesh Shader, Ray Tracing 확장성으로 나뉩니다. 아래 여섯 영역은 데이터 레이아웃 - RHI - 실행 모델 - 렌더링 - 확장 - 설계 원칙 순으로 이 방향에 대응합니다.

1) 데이터 지향(DOD, Data-Oriented Design) 메모리 레이아웃

렌더링 시뮬레이션의 반복 실행에서는 'Object'를 직접 순회하기보다, Transform (Position, Rotation), Physics (Velocity, Mass), Material (Index, Parameters)처럼 용도별 데이터를 별도의 플랫 배열로 분리합니다. 예를 들어 물리 갱신 시 CPU는 필요한 Velocity와 Mass 배열만 선형으로 읽으며, 이 구조는 캐시 라인 안에서 실제로 사용하는 데이터의 비율을 높입니다. 16바이트로 정렬된 연속 데이터는 SSE 기반 SIMD 처리에도 적합합니다.

2) RHI (Render Hardware Interface)

Vulkan, D3D12, Metal 같은 저수준 그래픽스 API를 얹고 투명하게 감싸는 계층입니다. 그래픽스뿐 아니라 향후 도입할 Compute Shader, Mesh Shader, Ray Tracing 커맨드까지 동일한 인터페이스에서 capability 단위로 노출하도록 설계합니다.

메커니즘	세부 기술	설계 목적
Thread-Local RHI	스레드별 독립 'ICommandList'와 할당자	병렬 커맨드 기록 시 전역 경합 제거
Resource Factory	'IBuffer'와 'ITexture' 핸들 추상화	백엔드 구현체 은닉과 모듈 격리
Capability 기반 확장	Graphics, Compute Shader, Mesh Shader, Ray Tracing 노출	고급 파이프라인 도입의 단일 진입점

- **IDevice (Factory):** 스왑체인 관리, 리소스 생성, 전역 동기화의 정점입니다. 모든 리소스는 이 팩토리로만 생성되며 백엔드 구현은 외부에 노출되지 않습니다.
- **ICommandList (Thread-Local):** 워커 스레드는 'AcquireCommandList()'로 전용 기록기를 확보합니다. 반복 호출이 많은 경로에서는 얇은 핸들 기반 호출이나 백엔드별 직접 호출 경로를 검토하며, 'DispatchCompute()', 'DrawMeshTasks()', 'DispatchRays()'는 로드맵에 따라 단계적으로 추가합니다.
- **Resource Management:** 'BufferUsage'와 'TextureUsage' 비트마스크로 용도를 정의해 GPU 힙(Upload/Default) 배치를 최적화합니다.
- **점진적 제어의 최하단:** RHI는 사용자가 직접 내려올 수 있는 가장 낮은 통제 계층입니다. 상위 렌더러의 기본값으로 시작해, 필요에 따라 'RendererDesc' config - 상위 렌더러의 저수준 API - 'IDevice', 'ICommandList' 직접 사용 순으로 원하는 깊이까지 제어를 가져갈 수 있습니다 (II 장에서 제시한 점진적 제어 강점의 설계 근거).

3) Multi-Threading 및 Task Scheduling

CPU의 모든 코어를 활용하기 위한 실행 모델로, 연산을 태스크 단위로 분산합니다. 로직 스레드와 렌더 스레드 간 데이터 경합을 막는 상태 다중화(State Separation)도 이 영역에 속합니다.

핵심 메커니즘	세부 기술	설계 목적
Task System	의존성 그래프 기반 Job 스케줄링	멀티코어 처리량 활용
병렬 커맨드 기록	Thread-local 'ICommandList'	수만 드로우 콜 동시 기록
State Separation	Frame Indexing과 버퍼 다중화	로직-렌더 스레드 동기화 락 제거
제출 순서 보장	위상 정렬 + Sort Key 병합	데이터 일관성과 결정적 출력

- **Task System (Job System):** 연산을 태스크로 분산하고 Culling→Sorting→Command Recording→Submit의 선행 관계를 의존성 그래프로 정의합니다.

- **상태 분리 (State Separation):** 'IDevice::GetCurrentFrameIndex()'로 프레임 번호(0,1,2)를 얻어 갱신이 빈번한 버퍼를 프레임 수만큼 생성합니다. 시물레이션 스레드가 현재 프레임의 버퍼를 갱신하는 동안 GPU는 이전 프레임의 버퍼를 읽도록 분리하고, 재사용 시점에는 fence로 GPU 사용 완료를 확인합니다.
- **병렬 커맨드 기록 및 순서 보장:** 스레드별 독립 메모리로 여러 코어가 동시에 기록하고, 최종 제출 시 위상 정렬로 선행 태스크 완료를 보장합니다. Sort Key를 기준으로 커맨드 리스트를 정렬하고 병합해 단일 배치로 제출합니다.

4) 렌더링 파이프라인 및 Render Graph

상위 렌더러와 하위 RHI를 잇는 데이터 통로입니다. 렌더 패킷 생성과 정렬, Bindless 바인딩으로 드로우를 처리하고, 패스 간 의존성, 리소스 수명, 배리어는 Render Graph가 자동 산출합니다.

핵심 메커니즘	세부 기술	설계 목적
Flattened Render Queue	64-bit Sort Key 패킷	상태 변경 최소화와 드로우 콜 최적화
Bindless 전략	디스크립터/argument 리소스 테이블 + 소량 상수 전달	슬롯 바인딩 감소와 인덱스 기반 리소스 참조
Render Graph	패스의 읽기/쓰기 리소스 선언	실행 순서와 배리어 자동화, Transient aliasing

- **Flattened Render Queue:** 정렬 가능한 평탄 패킷을 생성하며 Sort Key는 [Depth(24)|PSO ID(24)|Instance ID(16)]입니다. 불투명 객체는 Front-to-Back로 정렬해 Overdraw를 줄이고 동일 PSO 패킷을 묶어 상태 변경을 최소화합니다.
- **Bindless 전략:** 백엔드별 디스크립터/argument 리소스 테이블을 통해 리소스를 인덱스로 참조하고, 소량 데이터는 각 API에 맞는 상수 전달 경로로 주입합니다.
- **Render Graph:** 각 패스가 사용할 리소스를 읽기/쓰기로 선언하면 그래프가 실행 순서와 리소스 배리어/동기화 명령을 자동 산출합니다. G-buffer 등 일시적 리소스는 Transient 풀에서 aliasing 재사용하며, 3절의 멀티스레드 패스 기록과 결합됩니다.

5) GPU 연산 확장 (Compute Shader, Mesh Shader, Ray Tracing) — 확장 계획

명시적 API의 최신 파이프라인을 단계적으로 도입해 CPU 의존을 줄이고 처리량을 높입니다. (현재 미구현, 로드맵 Phase 4~6.)

핵심 메커니즘	세부 기술	설계 목적
Compute Shader	DispatchCompute, GPU 쉐딩과 포스트프로세싱	CPU 병목 완화와 대역폭 활용
Mesh Shader	Task→Mesh 파이프라인, meshlet	GPU-driven geometry, 드로우 콜 급감
Ray Tracing (도전)	BLAS/TLAS, ray query와 Ray Tracing 파이프라인	하이브리드 그림자, 반사, AO

- **Compute Shader:** Compute PSO와 ‘DispatchCompute()’를 추가해 프러스텀 쉐딩, 오클루전 쉐딩, tiled/clustered 라이트 쉐딩, bloom, tone mapping을 GPU에서 수행합니다.
- **Mesh Shader:** 고정 IA와 VS 단계의 일부 역할을 Task→Mesh 셰이더로 옮기고 지오메트리를 meshlet 단위로 분할하면, GPU 기반 쉐딩과 결합해 드로우 호출과 CPU 지오메트리 처리 부담을 줄일 수 있습니다.
- **Ray Tracing (도전 과제):** BLAS/TLAS 가속 구조를 빌드하고 Ray Tracing 파이프라인 또는 inline ray tracing으로 그림자, 반사, AO를 하이브리드로 처리하며, DXR과 ‘VK_KHR_ray_tracing’에 매핑합니다.

6) 설계 원칙

- **Header Isolation.** RHI 모듈은 플랫폼/백엔드 헤더(‘Windows.h’, ‘vulkan.h’)를 노출하지 않으며 구현은 Internal 클래스와 Pimpl로 은닉합니다.
- **Data-Only Communication.** 모듈 간 통신은 무거운 객체 대신 POD 구조체나 인덱스(ID)로 하며, 직렬화와 네트워크 전송에도 유리합니다.

*** 병목(Bottleneck) 시나리오**

병목은 단일 기능이 아니라 CPU 제출, GPU 실행, 메모리 접근 전 구간에서 발생하며, 각 설계 요소가 이를 분담해 해소합니다.

병목 지점	원인	대응 기능
CPU 드로우 콜 제출	단일 스레드 기록	Multi-Threading(병렬 기록)
GPU 파이프라인 스톨	과다하거나 중복된 배리어	Render Graph 자동 배리어 + PSO 베이킹
지오메트리 처리량	고정 IA와 과다 드로우 콜	Mesh Shader(meshlet)
썬 쉐딩 CPU 비용	CPU frustum/occlusion	Compute Shader GPU 쉐딩
메모리 대역폭과 단편화	산재된 데이터와 잦은 동적 할당	SoA 구조와 연속 배열 배치

요약하면 본 엔진은 단일 기능 최적화가 아니라 CPU 제출-GPU 실행-메모리 접근 전 구간의 병목을 각 설계 요소가 분담해 해소하는 구조를 지향합니다.

V. 개발 일정 및 역할

1. 기능별 진척 사항

기능	진척사항	진척률 (%)
Foundation 및 플랫폼	▪ SIMD 기반 벡터 · 행렬 수학 모듈	100%
	▪ 윈도우, 타이머 플랫폼 계층	
	▪ CMake 기반 백엔드 선택 빌드 구성	
RHI 공통 인터페이스	▪ IDevice · ICommandList 공통 인터페이스	75%
	▪ 버퍼 · 텍스처 · 파이프라인 · 동기화 객체 흐름	
	▪ Vulkan · D3D12 · Metal · Null 구현체 연결	
DirectX 12	▪ D3D12Device와 DXGI 스왑체인 구성	100%
	▪ D3D12PipelineState와 Root Signature 구성	
Vulkan 렌더링	▪ VulkanDevice와 스왑체인 구성	100%
	▪ spir-v 셰이더 컴파일과 VKCommandList 구성	
Metal 렌더링	▪ MTLDevice, CAMetalLayer, CommandQueue 구성	100%
	▪ MSL 셰이더 컴파일과 Render Pipeline 생성	
렌더링 기능	▪ Forward / Deffered 렌더링 경로 구성	70%
	▪ 재질, 텍스처, 조명 데이터를 렌더 패킷으로 전달	
	▪ 조명, 그림자, Post Processing 처리	
	▪ 렌더 패킷 바인딩 경로 구성	
모델 로더 및 에셋 처리	▪ OBJ, FBX, GLTF 포맷 로딩	80%
	▪ 좌표계, UV, 인덱스 후처리	
확장 기능 및 로드맵	▪ Render Graph 패스와 배리어 자동화 설계	20%
	▪ Compute Shader / Ray Tracing (도전)	
	▪ Mesh Shader	
	▪ CPU / GPU Profiler	

2.1 한재승-20242215

학번	20242215	이름	한재승
기여도	9 / 10점	기술수준	9 / 10점
구분	내용		
수행 내용	• RHI 인터페이스 설계 및 합의 Vulkan, D3D12, Metal 담당 팀원과 API 공통 계약을 정리한 후 RHI 설계. API의 직접 구현은 각 백엔드 담당이 수행, 공통 인터페이스와 계층 경계를 담당. • 데이터 레이아웃 및 수학 모듈 설계 SoA 구조를 고려한 데이터 배치 방향을 정리, SIMD 기반 벡터·행렬 모듈 설계. • 빌드 및 플랫폼 계층 구성 CMake 기반 크로스플랫폼 빌드 구성을 정리하고, API / SIMD 빌드 분기를 관리. • 엔진 코어 아키텍처 설계 렌더러 골격과 IRenderPath 기반 바인딩 전략 계층을 설계함.		
과제수행 시 문제점 및 해결방안	• RHI 계약 변경 비용 [문제] 최소 RHI로 시작, 요구 기능이 늘어나면서 인터페이스 갱신 비용이 커짐. [해결] 핵심 계약만 순수 가상(pure virtual) 메서드로 두고, 선택 기능은 기본 구현이 있는 옵션 메서드로 분리함. • 추상화 수준 경계 [문제] 조명, 그림자, 모델 로딩 기능을 RHI에 포함하면 인터페이스가 비대해지고 백엔드별 수정 범위가 커지는 문제가 있었음. [해결] RHI는 리소스 생성, 커맨드 기록, 동기화 같은 하드웨어 직접 제어 기능으로 제한하고, 조명과 모델 데이터 처리는 상위 렌더러/썬 계층으로 분리함. • 플랫폼 헤더 오염 [문제] Windows.h, vulkan.h 같은 플랫폼 헤더가 상위 모듈로 노출되어 빌드 문제와 바이너리 비대화를 발견. [해결] Pimpl 은닉을 적용해 플랫폼 헤더를 백엔드 구현 내부로 제한함. • 팀원 구현 코드 검토 및 통합 방향 결정 [문제] Light, Shadow, Model, Mesh 기능이 팀원별로 각각 구현되면서 데이터 전달 방식과 렌더링 경로 연결 방식이 서로 달랐음. [해결] 각 구현 코드를 검토한 뒤, 엔진의 RHI와 Renderer 구조에 가장 잘 맞는 방식을 선택하여 공통 렌더링 흐름에 맞게 통합 방향을 정리함.		

2.2 윤 훈-20240901

학번	20240901	이름	윤 훈
기여도	(9 / 10점)	기술수준	(9 / 10점)
구분	내용		
수행 내용	이번 과제에서 내가 맡은 부분은 Vulkan 브랜치의 하위 RHI 구현을 상위 렌더러 구조와 연결하고, 그 위에서 빛과 그림자가 실제 렌더링 결과로 보이도록 만드는 작업이었다. 처음에는 Vulkan 백엔드의 장치 초기화, 명령 기록, 파이프라인 생성, GPU 리소스 관리 흐름을 중심으로 화면 연결, 스왑체인, 렌더 패스, 그래픽스 파이프라인, 버퍼, 텍스처, 리소스 바인딩 구조를 구성하였다. CMake에서는 glslangValidator를 호출해 GLSL 셰이더를 SPIR-V로 변환하는 흐름을 작성했고, GLFW 기반 화면 연결까지 포함하여 Vulkan 백엔드가 독립적으로 프레임을 출력할 수 있는 기반을 만들었다. 기본 출력 경로가 잡힌 뒤에는 빛과 그림자를 상위 렌더단으로 올리는 작업을 진행하였다. 단순히 셰이더 안에 조명 값을 직접 넣는 방식으로는 상위 렌더러, 장면 데이터, 재질 정보, 렌더링 플래그와 자연스럽게 연결하기 어려웠기 때문에, 먼저 데이터 계약을 정리했다. 공통 셰이더 레이아웃을 분리하여 C++ 코드와 GLSL 셰이더가 같은 바인딩 번호, 텍스처 사용 플래그, 소량 상수 데이터 위치를 보도록 만들었고, 렌더러 설정값, 장면의 대표 조명 정보, 그림자 생성/수신 플래그, 그림자 맵 설정을 빛과 그림자 기능에 맞게 정리하였다. 이 과정에서 조명과 그림자 파라미터가 특정 셰이더 파일 내부에 머무르지 않고, 상위 렌더러가 제어하는 구조로 올라가게 되었다. 그림자 처리에서는 그림자 패스와 메인 렌더 패스를 분리했다. 그림자 패스에서는 그림자를 생성하는 draw만 광원 시점의 깊이 맵에 기록하고, 메인 렌더 패스에서는 그림자를 받는 draw만 픽셀 셰이더에서 그림자 맵을 샘플링하도록 구성하였다. 그림자 맵 계산 단계에서는 조명 방향 또는 위치를 기준으로 광원 시점 행렬을 계산하고, 이를 그림자 행렬 데이터로 셰이더에 전달하였다. 조명 셰이더는 Phong의 확산광/반사광 개념을 참고하여 시작했지만, 상위 Vulkan 셰이더에서는 GGX, Smith 기하 감쇠, Schlick Fresnel 기반의 간이 PBR 직접광 모델로 확장하였다. 덕분에 빛의 방향, 표면 거칠기, 금속도, 그림자 가시성이 한 셰이더 흐름 안에서 함께 반영되도록 만들 수 있었다.		

	리소스 바인딩 쪽에서는 텍스처, 조명 상수 데이터, 그림자 맵, 그림자 행렬 데이터, 정점/인덱스 저장 버퍼를 GPU 리소스 바인딩 묶음에 배치하고, draw별 변환 행렬과 재질 값은 소량 상수 데이터로 넘기도록 정리하였다. 일반적인 정점 입력 방식에만 의존하지 않고 저장 버퍼 기반의 셰이더 지오메트리 읽기 구조를 적용하여, 셰이더가 draw 정보를 기준으로 정점/인덱스 데이터를 직접 읽게 만들었다. 이 구조는 당장의 빛·그림자 구현뿐 아니라 이후 렌더 패킷, 재질 시스템, 다중 백엔드 확장까지 고려했을 때도 상위 렌더러와 Vulkan 백엔드 사이의 경계를 더 명확하게 만들어 주었다.
과제수행 시 문제점 및 해결방안	작업 중 가장 크게 부딪힌 문제는 처음 만든 빛과 그림자 코드가 특정 셰이더 파일과 상수 버퍼 구조에 너무 강하게 묶여 있었다는 점이었다. 화면에 그림자가 보이는 것 자체는 가능했지만, 그 상태에서는 상위 렌더러가 조명과 그림자 값을 안정적으로 제어하기 어려웠고, C++ 코드와 GLSL 셰이더의 바인딩 번호나 소량 상수 데이터 크기가 조금만 달라져도 렌더링 결과가 깨졌다. 이 문제를 해결하기 위해 공통 셰이더 레이아웃을 기준으로 만들고, C++ 구조체와 GLSL 데이터 규칙이 같은 값을 참조하도록 정리하였다. 이 변경 이후에는 조명/그림자 기능이 셰이더 파일 하나에 묶인 임시 구현이 아니라, 상위 렌더러와 Vulkan 백엔드가 공유하는 정식 렌더링 계약으로 자리 잡게 되었다. 두 번째 문제는 조명 값과 그림자 행렬이 프레임마다 갱신되는 과정에서 빛이 깜빡이거나 그림자 위치가 순간적으로 어긋나 보이는 현상이었다. Vulkan은 CPU와 GPU 동기화를 직접 관리해야 하기 때문에, 프레임 진행 순서와 리소스 바인딩 갱신 순서가 맞지 않으면 GPU가 아직 사용 중인 데이터를 CPU가 다시 갱신하는 상황이 생길 수 있다. 이 문제는 프레임 동기화 객체, 프레임별 이미지 사용 상태, 현재 프레임 기준의 리소스 사용 흐름을 정리하면서 해결하였다. 이후에는 조명 방향, 그림자 행렬, 리소스 바인딩 갱신 순서가 안정화되어 빛이 깜빡이는 증상이 사라졌고, 그림자도 프레임마다 같은 위치와 강도로 유지되었다. 세 번째 문제는 그림자 맵 특성상 self-shadowing, shadow acne, 경계 aliasing, 화면 밖 샘플링 문제가 동시에 나타났다는 점이다. 특히 바닥이나 모델 표면에 불필요한 얼룩처럼 보이는 그림자가 생기거나, 조명 방향이 바뀔 때 그림자 경계가 튀어 보이는 문제가 있었다. 이를 해결하기 위해 그림자 전용 패스를 따로 구성하고, 픽셀 셰이더의 깊이값 비교 과정에 고정 보정값, 기울기 기반 보정값, 법선 기반 보정값을 조합하였다. 또한 PCF 샘플링을 적용해 그림자 경계를 부드럽게 만들고, 그림자 맵

계산 단계에서는 장면 범위를 기준으로 광원 시점 투영 범위를 조정하여 그림자 맵 해상도가 불필요한 영역에 낭비되지 않도록 했다. 이 처리로 그림자 acne와 경계 떨림이 크게 줄었고, 최종 결과에서는 빛과 그림자가 훨씬 안정적으로 보이게 되었다.

마지막으로 보완해야 할 점도 분명했다. 현재 구조는 대표 방향광 또는 점광 중심으로 동작하고, 점광의 경우 모든 방향을 덮는 큐브 그림자가 아니라 단일 투영 기반 그림자 맵에 가깝다. 또한 GGX 기반 BRDF 계산은 들어갔지만 완전한 IBL 기반 PBR 파이프라인까지 구현한 것은 아니다. 따라서 이후에는 다중 조명 처리, 점광 큐브 그림자, 계단식 그림자 맵, 안정적인 그림자 투영, IBL 환경광까지 확장할 필요가 있다. 다만 이번 작업을 통해 Vulkan 백엔드에서 빛과 그림자를 상위 렌더링 경로에 올리는 기본 구조와 실제 문제 해결 흐름은 확보했다고 볼 수 있다.

2.3 정현진-20242207

학번	20242207	이름	정현진
기여도	(9 / 10점)	기술수준	(9 / 10점)
구분	내용		
수행 내용	프로젝트에서 저는 DirectX 12(D3D12)를 기반으로 하는 렌더링 백엔드 시스템 초기화 및 파이프라인 구성과, 이를 바탕으로 한 다중 포맷 3D 모델 로더 개발을 담당하였습니다. 먼저, D3D12 백엔드의 장치 초기화, 커맨드 리스트를 통한 명령 기록, 루트 시그니처 및 그래픽스 파이프라인 생성, 그리고 GPU 리소스 관리 흐름을 전반적으로 구축했습니다. 이를 통해 그래픽스 디바이스 설정부터 렌더링 명령 제출, 그리고 정점 및 텍스처 데이터의 그래픽 메모리 전송에 이르는 렌더링 백엔드의 핵심 기반을 독립적으로 완성했습니다. 다음으로, 구축된 환경 위에 다양한 3D 에셋을 띄우기 위해 OBJ, FBX, GLTF를 모두 지원하는 다중 포맷 모델 로더를 구현했습니다. 파일 파싱 과정에서 기하학적 형상 데이터는 메시(Mesh)로, 질감 및 표면 속성은 머터리얼(Material) 정보로 분리하여 효율적으로 리소스를 관리하고 이후 라이팅 연산에 활용될 수 있도록 시스템을 구성했습니다. 나아가 각 파일 규격의 특징을 반영하여 실제 파이프라인에 적합하도록 정점 데이터를 세부적으로 가공했습니다. ● OBJ 로더는 파일 스트림 기반의 자체 파서를 구현했으며, 다각형(N-Gon) 면을 팬 삼각분할(Fan Triangulation) 방식을 통해 삼각형 메시로 동적 변환했습니다. 추가로 외부 재질 파일(.mtl)을 직접 파싱해 기본 색상과 텍스처 경로를 추출했습니다. ● GLTF 로더는 파일 내 기본 씬(Default Scene)을 기준으로 씬 그래프를 순회하도록 구현했습니다. 각 노드의 로컬 변환 행렬을 누적해 정점과 법선 벡터에 미리 연산(Baking)하고, 좌표계 방향 및 UV 특성에 맞춰 최상위 노드를 Y축 기준으로 180도 회전 및 UV Y축 반전 처리를 적용했습니다. ● FBX 로더는 파싱 라이브러리(ufbx)를 활용해 다각형 메시지를 삼각화하고, 씬 구조에서 추출된 각 정점에 노드의 전역 변환 행렬을 적용하여 렌더링 시 모델이 정확한 월드 공간 위치에 배치되도록 처리했습니다. 마지막으로, 우리 엔진이 D3D12, Vulkan, Metal 등 다중 그래픽스 API를 지원한다는 점		

	을 고려하여 데이터 추상화 작업을 진행했습니다. 로더에서 읽어 들인 다양한 형태의 모델 정점 데이터를 엔진 내부용 공통 표준 구조로 일괄 변환함으로써, 어떤 백엔드 환경에서도 동일한 규격으로 그래픽스 처리가 가능하도록 통합 데이터 기반을 마련했습니다.
과제수행 시 문제점 및 해결방안	과제를 수행하며 D3D12 파이프라인의 리소스 바인딩 복잡성과 데이터 정규화 과정에서 문제점에 직면했습니다. 먼저, D3D12 파이프라인을 구축할 때 버퍼(Buffer)와 텍스처(Texture) 같은 리소스를 렌더링 단계에서 셰이더에 연결(Binding)하는 과정의 복잡성 문제가 있었습니다. 기존 방식으로 리소스마다 디스크립터 테이블을 구성하고 렌더 루프 내에서 매번 바인딩 상태를 교체하는 것은 비효율적이며 코드가 매우 복잡해지는 단점이 있었습니다. 이를 해결하기 위해 엔진 내부적으로 단일 전역 디스크립터 힙(Global Descriptor Heap)을 유지하는 관리 구조를 도입했습니다. 리소스가 생성될 때 동적으로 슬롯 인덱스(AllocateDescriptorSlot)만 발급받고, 렌더링 시에는 해당 인덱스들을 푸시 상수(Push Constants)로 셰이더에 한 번에 전달하여 셰이더 내부에서 리소스에 직접 접근하도록 하는 바인드리스(Bindless) 스타일의 접근법을 채택했습니다. 결과적으로 파이프라인의 바인딩 로직이 획기적으로 단순화되고, 향후 렌더링 성능을 최적화할 수 있는 강력한 기반을 마련했습니다. 다음으로, 모델 로더를 구현하면서 3D 모델 포맷별로 사용하는 공간 좌표계와 정점 데이터의 인덱싱 방식이 상이하여 발생하는 데이터 파편화 문제를 겪었습니다. 별도의 처리 없이 데이터를 렌더링 파이프라인에 넘길 경우 모델이 뒤집히거나 표면의 법선 벡터가 꼬여 조명 연산이 비정상적으로 수행되었습니다. 이를 해결하기 위해 파일 파싱을 완료하는 시점에 축 변환 연산과 인덱스 순서를 엔진의 렌더링 기준에 맞춰 강제로 재정렬하는 후처리 로직을 추가했습니다. 결과적으로 렌더러가 파일 원본 포맷과 무관하게 통일된 렌더링 연산을 수행할 수 있게 되었습니다. 보완해야 할 부분도 있습니다. 현재 구현된 모델 로더 시스템은 단일 오브젝트나 중간 규모의 에셋을 렌더링하는 데에는 안정적으로 동작하지만, 대규모 씬(Scene) 데이터나 고해상도 모델을 로드할 때 몇 가지 기능적, 성능적 한계를 지니고 있어 향후 개선이 필요합니다. 먼저, GLTF 로더의 다중 씬(Multi-Scene) 지원 부재입니다. 현재 로더의 구현체는 파일

내에 여러 개의 씬 데이터가 존재하더라도, 명시된 기본 씬(Default Scene) 하나만 추출하여 파싱하도록 하드코딩되어 있습니다. 이로 인해 다수의 씬으로 구성된 복잡한 배경 맵이나 전체 레벨(Level) 에셋을 한 번에 불러오지 못하는 제약이 있습니다. 향후 파서를 확장하여 전체 씬 그래프 구조를 순회하고 개별 씬을 독립된 인스턴스로 관리할 수 있는 기능을 추가해야 합니다.

다음으로, 대규모 모델 파일 파싱 시의 로딩 병목 및 메모리 최적화 문제입니다. 현재 GLTF와 FBX 로딩 단계에서는 노드의 계층적 전역 변환 행렬(Transform Matrix)을 모든 개별 정점 위치와 법선에 일일이 곱해서(Baking) 파이프라인에 넘겨주는 방식을 사용하고 있습니다. 이 방식은 파이프라인 처리를 단순하게 만들어 주지만, 수백만 개의 폴리곤을 가진 대용량 에셋을 로드할 때는 행렬 곱셈 연산으로 인해 파싱 속도가 저하되고 메모리 사용량이 급증하는 원인이 됩니다. 이를 보완하기 위해 향후에는 정점 데이터를 로컬 상태 그대로 GPU 버퍼에 올리고, 계층적 변환 행렬 정보를 상수 버퍼(Constant Buffer)나 구조화 버퍼를 통해 전달하여 셰이더(Shader) 단계에서 런타임에 연산하도록 로더와 렌더러 구조를 최적화할 계획입니다.

2.4 정준혁-20240846

학번	20240846	이름	정준혁
기여도	(9 / 10점)	기술수준	(9 / 10점)
구분	내용		
수행 내용	이번 과제에서 제가 맡은 부분은 Metal 백엔드에서 GPU 리소스가 생성되고 관리되는 흐름을 이해하고, 이를 엔진의 상위 RHI 구조와 연결하는 것이었습니다. Metal은 Objective-C 기반의 그래픽 API이고, 저희 엔진은 C++ 기반으로 구성되어 있기 때문에 두 구조를 직접 섞기보다는 MetalDevice, MetalBuffer, MetalTexture, MetalCommandList 같은 클래스를 통해 Metal 객체를 감싸는 방식으로 정리했습니다. 이를 통해 상위 렌더러에서는 Metal API를 직접 호출하지 않고 공통 RHI 인터페이스만 사용하면서도, 내부적으로는 Metal 리소스를 생성하고 사용할 수 있도록 구성했습니다. 특히 Metal의 자동 메모리 관리 구조를 중심으로 버퍼와 텍스처의 수명 관리 방식을 분석했습니다. Metal 객체는 MTLBuffer, MTLTexture, MTLCommandBuffer처럼 Objective-C 객체로 관리되며, ARC를 사용할 경우 객체의 참조 관계에 따라 retain과 release가 자동으로 처리됩니다. 저는 이러한 구조를 C++ 코드와 연결하기 위해 Impl 구조체 내부에 Metal 객체를 보관하고, 외부에는 공통 인터페이스만 노출하는 방식으로 구현 흐름을 정리했습니다. 이 방식은 Metal 객체가 C++ 코드 전체에 직접 퍼지는 것을 막아 주고, 리소스 관리 책임을 각 래퍼 클래스 안에 모을 수 있다는 장점이 있습니다. 버퍼 관리 부분에서는 CPU와 GPU가 데이터를 공유할 수 있도록 Shared 메모리 방식을 사용했습니다. MetalBuffer에서 버퍼를 생성할 때 MTLResourceStorageModeShared 옵션을 적용하여 CPU에서 작성한 데이터를 GPU가 사용할 수 있도록 했고, Map 함수에서는 버퍼의 메모리 주소를 반환하도록 구성했습니다. 이 구조를 사용하면 CPU에서 정점 데이터나 상수 데이터를 갱신할 때 별도의 복잡한 복사 과정 없이 GPU가 같은 메모리 영역을 참조할 수 있습니다. 따라서 작은 규모의 렌더링 데이터나 자주 갱신되는 데이터에 적합한 방식이라고 판단했습니다. 텍스처 관리 부분에서는 엔진에서 사용하는 TextureDesc 정보를 Metal의 텍스처 생성 정보로 변환하는 과정을 확인했습니다. 텍스처의 너비, 높이, 포맷, mip level, 사용 목적을 MetalTextureDescriptor에 맞게 설정하고, ShaderRead, RenderTarget, ShaderWrite 같은 사용 목적을 Metal의 텍스처 usage 값으로 변환했습니다. 또한 이미지 초기 데이터가 존재하는 경우 replaceRegion을 사용해 CPU 메모리에 있는 픽셀 데이터를 Metal 텍스처 메모리로 업로드했습니다. 이를 통해 상위 렌더러에서 장면		

	텍스처를 생성하면 Metal 백엔드가 실제 GPU 텍스처를 만들고 셰이더에서 사용할 수 있도록 연결했습니다. Renderer 쪽에서는 장면에 필요한 텍스처를 매 프레임 새로 생성하지 않고, 한 번 생성한 뒤 SceneTextureState에 저장하여 재사용하는 흐름을 확인했습니다. 텍스처가 처음 필요할 때만 CreateTexture를 호출하고, 이후에는 descriptor index를 통해 해당 텍스처를 다시 참조하도록 구성했습니다. 종료 시점에는 DestroyTexture와 DestroyPipelineState를 호출하여 생성된 GPU 리소스를 정리했습니다. 이 과정에서 Metal의 자동 메모리 관리 구조와 C++ 객체의 명시적 해제 구조가 함께 동작해야 한다는 점을 이해할 수 있었습니다.
과제수행 시 문제점 및 해결방안	과제를 진행하면서 가장 어려웠던 점은 C++ 객체와 Metal 객체의 메모리 관리 방식이 서로 다르다는 점이었습니다. C++에서는 new로 생성한 객체를 delete로 직접 해제해야 하지만, Metal에서 사용하는 객체들은 Objective-C 객체이기 때문에 ARC를 사용할 경우 참조 관계를 기반으로 자동 관리됩니다. 처음에는 이 두 방식이 섞이면 어느 쪽에서 리소스를 해제해야 하는지 헷갈렸습니다. 이를 해결하기 위해 Metal 객체는 Impl 구조체 내부에 보관하고, C++ 클래스의 생성자와 소멸자에서 Impl의 수명을 관리하는 방식으로 구조를 분리했습니다. 이렇게 하니 상위 코드에서는 C++ 객체처럼 사용할 수 있고, 내부에서는 Metal 객체의 참조 관리 흐름을 따를 수 있었습니다. 두 번째 문제는 bridge 사용 방식이었습니다. Metal 객체를 C++ 코드에서 void 포인터 형태로 넘겨야 하는 경우가 있었는데, 이때 단순히 포인터를 변환하는 것과 객체의 소유권까지 넘기는 것은 다르다는 점을 확인했습니다. 소유권 이전을 잘못 처리하면 객체가 너무 빨리 해제되거나 반대로 해제되지 않아 메모리 누수가 발생할 수 있습니다. 이 문제를 해결하기 위해 단순 참조가 필요한 경우에는 소유권을 넘기지 않는 bridge 방식을 사용하고, 실제 객체의 생성과 보관은 Metal 래퍼 클래스 내부에서 담당하도록 정리했습니다. 세 번째 문제는 GPU 리소스의 해제 시점이었습니다. CPU 객체는 필요 없어지면 바로 삭제할 수 있지만, GPU는 커맨드 버퍼에 기록된 작업을 나중에 실행할 수 있기 때문에 리소스를 너무 빨리 삭제하면 문제가 생길 수 있습니다. 예를 들어 렌더링 명령에 사용된 텍스처나 버퍼가 GPU에서 아직 사용 중인데 CPU 쪽에서 먼저 해제하면 렌더링 결과가 깨지거나 오류가 발생할 수 있습니다. 현재 구조에서는 렌더러 종료 시점에 텍스처와 파이프라인을 정리하도록 했고, 향후에는 프레임 인덱스를 기준으로 몇 프레임 뒤에 리소스를 해제하는 지연 삭제 큐를 도입하면 더 안전한 구조가 될 수 있다고 판단했습니다. 네 번째 문제는 자동 메모리 관리가 제대로 적용되기 위해 빌드 설정도 함께 확인해야

	한다는 점이었습니다. Metal 파일은 Objective-C++ 파일로 컴파일되어야 하며, ARC를 사용할 경우 컴파일 옵션에서 이를 명확하게 지정하는 것이 중요합니다. 현재 구조에서는 Metal 백엔드 파일을 Objective-C++로 컴파일하는 흐름은 구성되어 있지만, ARC 사용 여부를 명확히 관리하지 않으면 Metal 객체의 자동 해제 흐름이 불안정할 수 있습니다. 따라서 이후 보완할 점으로 fobjc-arc 옵션을 명시하거나, ARC를 사용하지 않는 경우에는 생성된 Metal 객체에 대한 release 처리를 명확하게 추가해야 한다고 정리했습니다. 마지막으로 보완해야 할 점도 있습니다. 현재 구조는 기본적인 버퍼, 텍스처, 커맨드 버퍼 관리에는 적합하지만, 실제 엔진 규모가 커지면 리소스가 많아지고 프레임마다 생성과 해제가 반복될 수 있습니다. 따라서 이후에는 Metal Heap을 이용한 리소스 묶음 관리, 프레임별 임시 리소스 관리, GPU 작업 완료 후 안전하게 해제하는 지연 삭제 구조까지 확장할 필요가 있습니다. 이번 작업을 통해 Metal의 자동 메모리 관리 방식과 C++ 엔진의 명시적 수명 관리 방식을 함께 사용하는 기본 구조를 이해하고, GPU 리소스를 안정적으로 관리하기 위한 방향을 잡을 수 있었습니다.
--	--

VI. 참고자료

번호	종류	제목	저자	발행연도
1	서적	Real-Time Rendering, 4th Edition	T.Akenine-Moller 외	
2	서적	DirectX12를		
3	문서	Vulkan Specification	Khronos Group	
4	컨퍼런스	CppCon 2014 “Data-Oriented Design and C++”	Mike Acton	2014
5	컨퍼런스	CppCon 2018 “OOP is Dead, Long Live Data-oriented Design”	Stoyan Nikolov	2018
6	컨퍼런스	CppCon 2025 “Cache-Friendly C++”	Jonathan Müller	2025
7	논문	Illumination for Computer Generated Pictures	Bui Tuong Phong	1975
8	온라인 문서	LearnOpenGL - Basic Lighting	Joey de Vries	온라인
9	온라인 문서	LearnOpenGL - Shadow Mapping	Joey de Vries	온라인
10	온라인 문서	LearnOpenGL - PBR Theory	Joey de Vries	온라인
11	공식 문서	Vulkan Tutorial - Frames in Flight	Khronos Group	온라인
12	공식 문서	Vulkan Specification - Resource Descriptors / Render Pass	Khronos Group	온라인
13	서적	Game Engine Architecture	Jason Gregory	2023
14		<i>What Every Programmer Should Know About Memory</i>	Drepper, U	2007
		<i>Pitfalls of Object-Oriented Programming</i> . Game Connect: Asia Pacific (GCAP), Sony Computer Entertainment	Albrecht, T	2009
